

Instituto Politécnico Nacional

Escuela Superior de Cómputo



Análisis y Diseño de Algoritmos

Práctica:

Programación Dinámica.

Fecha: 04/12/25

Domínguez Jiménez Ana Andrea

1. Ejercicios de programación.

Implementación del algoritmo de cambio de monedas.

a) Realice una estructura llamada *Cambio* en donde declaro las denominaciones (*d*) y la cantidad de monedas (*n*), y utilice la librería *limits.h* de c para utilizar la constante *INT_MAX* que permite limitar la cantidad almacenada en un int en bits y representa nuestro *INFINITO*.

```c
int main(){
    Cambio *C = (Cambio*)malloc(sizeof(Cambio));
    int monto;

    printf("Cuantas monedas desea ingresar? ");
    scanf("%d", &C->n);

    C->d = (int*)malloc(C->n * sizeof(int));

    printf("Ingrese las denominaciones:\n\n");
    for(int i = 0; i < C->n; i++) {
        printf("Moneda %i: ", i + 1);
        scanf("%i", &C->d[i]);
    }

    printf("\nCantidad total a cambiar: ");
    scanf("%d", &monto);

    int P[monto+1];
    int result = CHANGE(monto, C->d, C->n,P);

    if (result == INFINITO){
        printf("\nNo hay forma de formar ese monto.\n");
    }else{
        printf("\nMinimo numero de monedas: %d\n", result);

        printf("Monedas a devolver:\n");
        int i=monto;
        while(i>0){
            printf("- %d\n", P[i]);
            i-=P[i];
        }
    }
```

b) Dando como resultado el cambio mínimo de monedas y la denominación de ellas, El algoritmo calcula primero el cambio mínimo usando los arreglos D[] y P[], en while lo que hace es

restar para encontrar la solución óptima y la almacena en P[], la ultima denominación de moneda que se utilizo.

```
int CHANGE(int N, int d[], int n, int P[]){
    int D[N + 1];
    D[0] = 0;
    P[0]=0;

    for(int i = 1; i <= N; i++){//cambio
        D[i] = INFINITO;//valor inicial

        for(int j = 0; j < n; j++){ //cantidad de denominaciones
            if(i - d[j] >= 0 && D[i - d[j]] != INFINITO){

                int cambio = 1 + D[i - d[j]];
                if(cambio < D[i]){
                    D[i] = cambio;
                    P[i]=d[j];//minimo
                }
            }
        }
    }

    return D[N];
}
```

Eso se hace en la función *CHANGE* donde se encargará de validar el monto final de cambio.

c)

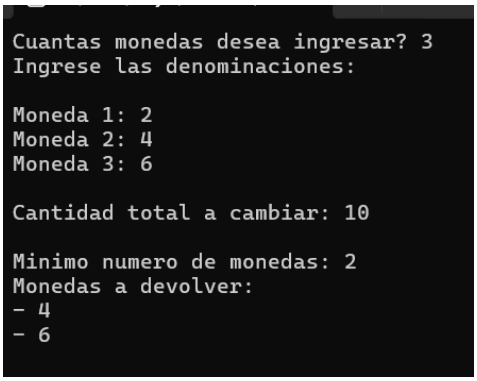
```
Cuantas monedas desea ingresar? 4
Ingrese las denominaciones:

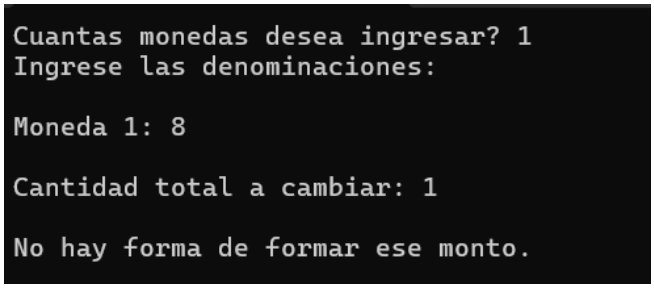
Moneda 1: 1
Moneda 2: 3
Moneda 3: 7
Moneda 4: 10

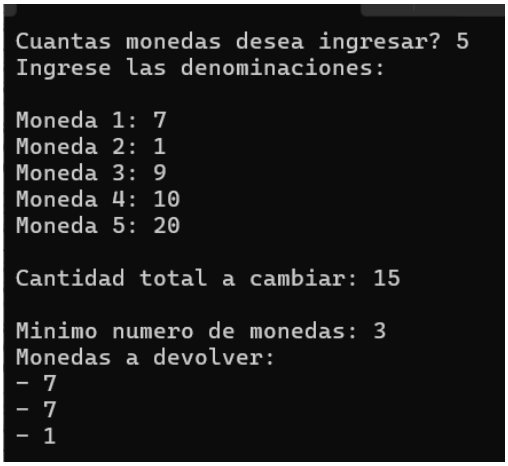
Cantidad total a cambiar: 15

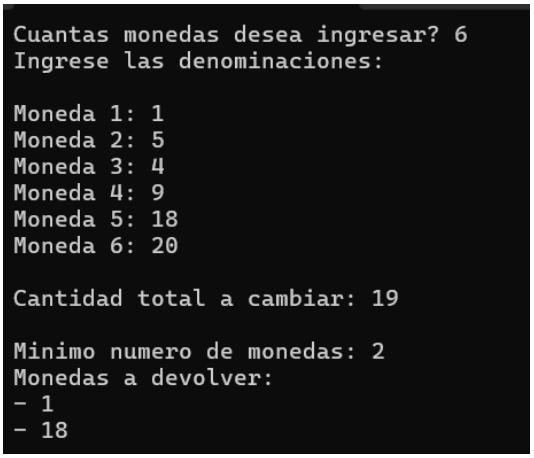
Minimo numero de monedas: 3
Monedas a devolver:
- 1
- 7
- 7


--------------------------------
```









2. Problema de la mochila fraccionaria.

Entrada:

Peso Mochila= W

objetos = n

Peso del objeto i= wi

Valor del objeto i= vi

Salida:

Que objetos pueden se tomados: obj[i].peso y obj[i].valor

Y cuando asi se requiera, Fraccion del objeto que se tomo:

T =W / obj[i].peso

Suma de los valores de los objetos tomados en la mochila: m

a)

```c
int main(){
    int W, n;
    printf("Ingrese la capacidad de tu mochila: ");
    scanf("%d", &W);

    printf("Cantidad de objetos: ");
    scanf("%d", &n);

    int vi[n], wi[n];

    for(int i=0; i<n; i++){
        printf("\nPeso Objeto %d: ",i+1);
        scanf("%d", &wi[i]);
        printf("Valor Objeto %d: ", i+1);
        scanf("%d", &vi[i]);
    }

    Mochila(W,n,wi,vi);
```

b)

```c
void Merge(Objeto A[],int n, Objeto L[], Objeto R[]){
void MergeSort(Objeto A[], int n){//mayor
void Mochila(int W, int n, int wi[], int vi[]){
    Objeto obj[n];

    for(int i=0; i<n; i++){
        obj[i].peso=wi[i];
        obj[i].valor=vi[i];
        obj[i].S=(float)vi[i]/wi[i];
    }
    MergeSort(obj,n);

    float m=0.0;
    printf("Objetos tomados: ");
    for(int i=0; i<n && W>0;i++){
        if(obj[i].peso <= W){
            W -=obj[i].peso; //se actualiza la capacidad de la mochila
            m+= obj[i].valor; //valor ganado
            printf("\n\nObjeto %i tomado ENTERO, Peso: %d | Valor: %d", i+1, obj[i].peso, obj[i].valor);1//cupo
        }else{
            float T=(float)W/obj[i].peso;
            m+=obj[i].valor*T;
            printf("\nObjeto %i tomando una parte, Peso: %d | Valor: %d", i+1, obj[i].peso, obj[i].valor, T);
            W=0;//mochila llena
        }
    }

    printf("\nSuma de los valores de los objetos tomados en la mochila: %.2f", m);
}
```

c)

```
Ingrese la capacidad de tu mochila: 10
Cantidad de objetos: 3

Peso Objeto 1: 10
Valor Objeto 1: 60

Peso Objeto 2: 20
Valor Objeto 2: 100

Peso Objeto 3: 30
Valor Objeto 3: 120
Objetos tomados:

Objeto 1 tomado ENTERO, Peso: 10 | Valor: 60
Suma de los valores de los objetos tomados en la mochila: 60.00
--------------------------------
```

```
Ingrese la capacidad de tu mochila: 5
Cantidad de objetos: 5

Peso Objeto 1: 1
Valor Objeto 1: 1

Peso Objeto 2: 2
Valor Objeto 2: 3

Peso Objeto 3: 4
Valor Objeto 3: 5

Peso Objeto 4: 2
Valor Objeto 4: 3

Peso Objeto 5: 5
Valor Objeto 5: 2
Objetos tomados:

Objeto 1 tomado ENTERO, Peso: 2 | Valor: 3

Objeto 2 tomado ENTERO, Peso: 2 | Valor: 3
Objeto 3 tomando una parte, Peso: 4 | Valor: 5
Suma de los valores de los objetos tomados en la mochila: 7.25
--------------------------------
```

```
Ingrese la capacidad de tu mochila: 100
Cantidad de objetos: 4

Peso Objeto 1: 8
Valor Objeto 1: 6

Peso Objeto 2: 1
Valor Objeto 2: 9

Peso Objeto 3: 5
Valor Objeto 3: 10

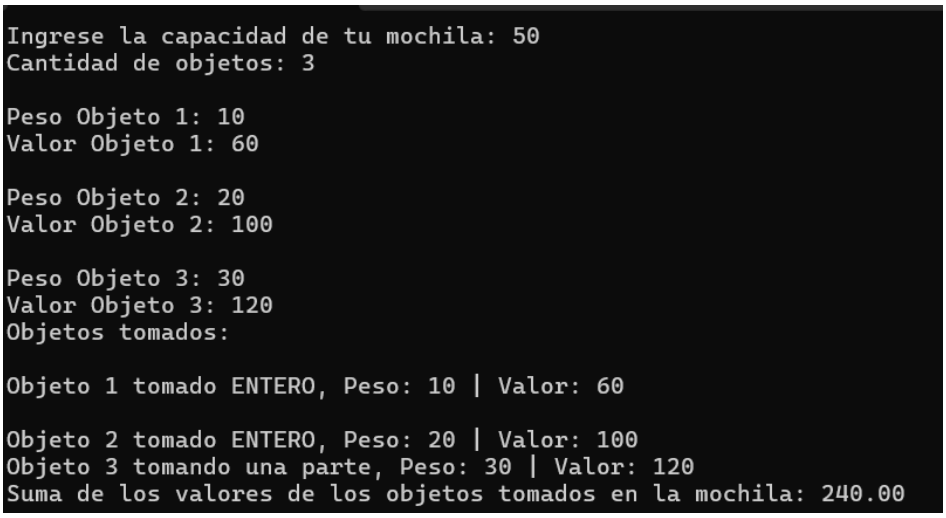
Peso Objeto 4: 7
Valor Objeto 4: 54
Objetos tomados:

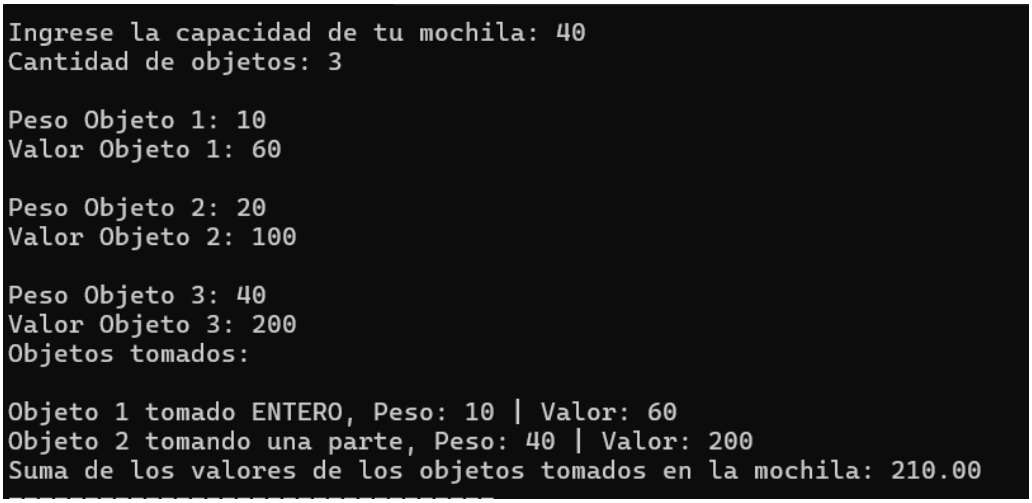
Objeto 1 tomado ENTERO, Peso: 1 | Valor: 9

Objeto 2 tomado ENTERO, Peso: 7 | Valor: 54

Objeto 3 tomado ENTERO, Peso: 5 | Valor: 10

Objeto 4 tomado ENTERO, Peso: 8 | Valor: 6
Suma de los valores de los objetos tomados en la mochila: 79.00
--------------------------------
```





d) Cada iteración es igual a O(n), Mergesort es O(nlogn) y es el que hace todo el trabajo mas complicado de ordenar por lo tanto la complejidad del programa es de **O(nlogn).**